

Problem: Dynamic Logic Circuit Simulation

Introduction to Logic Circuits

A logic circuit is a digital system composed of logic gates and wires, used to process binary signals (0 and 1). Signals propagate unidirectionally in the circuit: from input ports or the outputs of logic gates to the inputs of logic gates, and never in reverse.

The output value of a logic gate depends on its input signals, and a definite output can be computed only when all of its input signals are determined. This dependency forms a ***directed acyclic graph (DAG)***, where nodes represent input ports or logic gates, and edges represent signal flow.

In the circuit, the output of a node may serve as input to multiple logic gates, but each input pin of a logic gate can be connected to only one source node.

The basic logic gates are:

- ***AND gate***: Outputs 1 if and only if all inputs are 1.
- ***OR gate***: Outputs 1 if and only if at least one input is 1.
- ***XOR gate***: Outputs 1 if and only if the two inputs are different (one is 0, and the other is 1).
- ***NOT gate***: Outputs 1 if and only if the input is 0 (logical negation).

This problem requires simulating a dynamically changing combinational logic circuit. The circuit must support modifying connections, adding gates, and deleting gates, and after each operation, output the signal values of all specified output ports.

Problem Description

You are given an initial combinational logic circuit consisting of ***N*** nodes and their connections. Nodes are numbered starting from 0. Each node is either an input port (`INPUT`) or a logic gate (`GATE` , with a specified type). The signal value of each input port is given at definition time.

You are also given a set of output ports (nodes whose output values must be reported; they may be either input ports or logic gates).

There are ***T*** operations. Each operation is one of the following three types:

1. ***Add Logic Gate***

Create a new logic gate and specify its type and input sources.

Format:

```
ADD type [inputs...]
```

- For `AND`, `OR`, `XOR`: two inputs are required, e.g.

```
ADD AND src0 src1
```

- For `NOT`: one input is required, e.g.

```
ADD NOT src0
```

- `src0/src1`: the ID of the source gate

The new gate's ID is ****(current maximum node ID + 1)****.

The input guarantees that all source nodes exist.

1. ****Delete Logic Gate****

Delete an existing logic gate.

Format:

```
DELETE gate_id
```

- `gate_id`: the ID of the logic gate

It is guaranteed that:

- The logic gate exists.
 - Its output is not used by any other logic gate.
 - It is not one of the output ports.
-

1. ****Modify Connection****

Change one input pin of a logic gate to connect to another node.

Format:

```
MODIFY gate_id input_index new_src
```

- `gate_id`: the ID of the logic gate
- `input_index`: the input pin index (0 or 1; for a NOT gate it can only be 0)
- `new_src`: the ID of the new source node

Before applying the modification, you must check whether it would introduce a cycle in the circuit.

- If a cycle is formed, ****ignore the operation****, keep the circuit unchanged, and output a single line: ****Loop****
 - Otherwise, perform the modification.
-

After each operation:

- For every ****successfully executed**** operation (including successful `MODIFY`, `ADD`, and `DELETE`), recompute the output values of all output ports and print them in one line, in the

original output order, separated by spaces.

- For a `MODIFY` operation that would create a cycle, print only: ****Loop**** (Do not print signal values.)
-

Input Format

The first line contains three integers:

```
N K T
```

- `N`: number of initial nodes
- `K`: number of output ports
- `T`: number of operations

The next ****N**** lines describe the nodes:

- If the node is an input port:

```
INPUT value
```

where `value` is 0 or 1.

- If the node is a logic gate:

```
GATE type src0 src1
```

for `AND`, `OR`, `XOR` gates

```
GATE NOT src0
```

for `NOT` gates

It is guaranteed that:

- Every referenced source node ID exists.
- The initial circuit is acyclic.

The next line contains ****K**** integers, representing the IDs of the output ports, whose values must be printed in this order.

The next ****T**** lines each describe one operation, in the formats defined above.

Output Format

For each operation, output one line:

- If the operation is `MODIFY` and would create a cycle, output: ****Loop****
 - Otherwise, output a line containing ****K**** integers — the signal values of all output ports, in the given order, separated by spaces.
-

Sample Input

```
4 1 5
INPUT 0
INPUT 1
GATE AND 0 1
GATE NOT 2
3
ADD NOT 0
MODIFY 2 0 4
MODIFY 3 0 4
DELETE 2
MODIFY 3 0 3
```

Sample Output

```
1
0
0
0
Loop
```

Data Range

- $1 \leq N \leq 10000$
- $1 \leq K \leq 200$
- $1 \leq T \leq 10000$